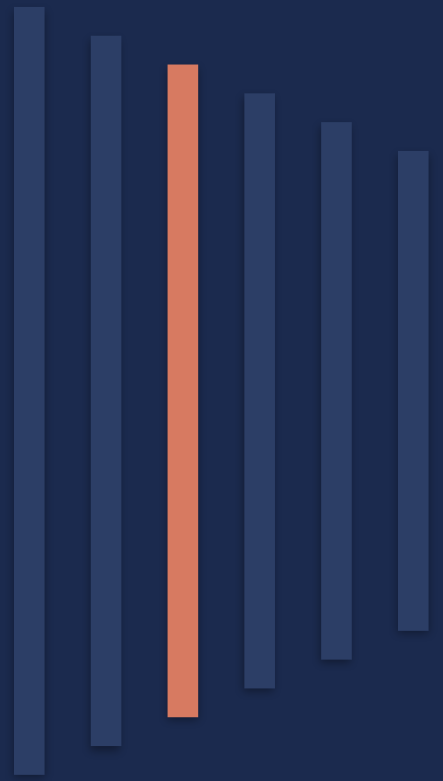
LESSON 02

Claude Code × Cursor

高级技巧

让工具替你做更多事，而不是你不停指挥工具。

Subagents | Hooks | Composer | Plugins



约 30 分钟 · 四件事 · 一杯茶时间

AGENDA · 今日路线图

四件事，递进展开

01

Subagents

派分身干脏活，
主线不污染、不刷屏

01 · 探索 / 审阅 / 研究

02

Hooks

把每次都得做的事
变自动执行

02 · 三事件 + 自动闭环

03

Cursor 进阶

Composer · Background
· 分层 Rules

03 · 跨文件改造 + 规则分层

04

Plugins

装一个，多十种思维范式

04 · ECC / Superpowers

课后还有：① bash 命令记录 hook ② subagent 审阅自己的代码

01

Subagents

派分身干脏活，主线只接收摘要。

隔离上下文做探索 / 审阅 / 研究，不污染主线。

Claude 干 20 分钟之后会变笨

✕ 不用 subagent 时

- **读 50 个文件之后**，它对最初的目标已经记不清
- **主线对话被刷屏** —— 几十次 Read / Grep 调用堆叠
- **所有源代码原文**都灌进主线上下文
- **下一轮想干新事**，已经被旧上下文拖慢

✓ Subagent 解决什么

- **把任务派到全新上下文** 独立子对话，不继承主线
- **结束后只把结论返回** 主线只看到一个 Task 折叠框
- **源码 / 中间产物不外溢** 主线上下文继续清爽
- **可以并行多派几个** 探索 + 审阅 + 研究 同时跑

主线 = 战术指挥；Subagent = 派出去执行的小队。

01 · 动手做 (3 分钟)

派 explore subagent 摸底 lesson1/demo/

STEP 1

在仓库根启动 Claude Code

```
cd <ai_lesson 仓库>
claude
```

STEP 2

整段粘贴这条 prompt 到主线

```
派一个 explore subagent 去摸底 lesson1/demo/ 的结构，
返回 10 行以内的摘要，
不要贴源码原文。
```

STEP 3

屏幕上你会看到一个折叠框

```
Task(subagent_type=Explore)
└ [一段 8-10 行的摘要，
  文件清单 + 结构判断]
```

观察三点： ① 主线只有一个 Agent 调用框 ② 摘要里没有源代码原文 ③ 主对话上下文没被几千 token 污染

01 · 三种典型用法

不是只有审阅：探索 / 审阅 / 研究

EXPLORE 探索

代码库摸底，主线不被刷屏

用例	进陌生仓库的第一件事
入口	prompts/explore-codebase.md
产出	8-10 行结构摘要

REVIEW 审阅

独立第二意见，避免自评偏高

用例	你刚写完一段代码
入口	prompts/review-diff.md
产出	风险列表 + 严重等级

RESEARCH 研究

外部信息汇总，主线不被参考资料淹没

用例	选型对比 / 文档调研
入口	prompts/research-libs.md
产出	结论 + 引用清单

范例 prompt 都在 lesson2/demo/01-subagents/prompts/ 下，复制即用。

Prompt 必须自包含

Subagent 看不到主线上下文 —— 目标 / 背景 / 重点 / 输出格式 全部写进去。

含糊不清

「审阅刚才那个函数」

→ Subagent 不知道「刚才」是哪个

自包含

```
审阅 src/auth/login.ts 中的 verifyToken。
背景：JWT 校验入口，被 5 个 API 调用。
重点：时间常数比较 / 过期时间 / 错误信息泄露。
输出：风险列表 + 严重等级。
```

三个反模式

- **派去做顺手就能干的小事**
  分身启动开销 > 自己干，得不偿失
- **把 subagent 当「省 token」工具**
  返回 200 行垃圾摘要不如自己读
- **Prompt 里写「刚才那个函数」**
  分身没有「刚才」的概念，靠猜会跑偏

团队级：把常用角色固化进 .claude/agents/<name>.md，提交进 git，全队共享。

02

Hooks

把每次都得做的事变自动执行。

.claude/settings.json 配一段， AI 不能跳过。

Hook = 在 X 时机被强制执行的 shell 命令

ENTRY

SessionStart

会话启动

为新会话注入项目状态，让 AI 一上来就知道现状

典型用途

`git status`
+ 当前分支
+ 最近 3 次提交

PRE

UserPromptSubmit

用户提交 prompt 前

在 AI 看到 prompt 之前预处理输入 / 注入项目状态

典型用途

校验 `prompt`
注入 `.env` 摘要
动态扩 `@` 引用

POST

PostToolUse

工具调用结束后

Edit 完跑 lint / Bash 完记日志 / 触发自动修复闭环

典型用途

`ruff --fix`
命令日志
类型检查

完整三事件示例：lesson2/demo/02-hooks/settings.json

02 · 动手做 (2 分钟)

验证 SessionStart 已生效

本仓库已经把三事件 hook 配好了，新开会话即可看到效果。

操作步骤

1. **新开一个 Claude Code 会话**

   hook 只在会话启动那一刻加载，旧会话不回溯生效

2. **在仓库根目录启动**

   ```
   cd <ai_lesson 仓库>
   claude
   ```

3. **看启动横幅下的 hook 输出**

   stdout 会被注入 Claude 的上下文 →
   下一段就能看到

应当看到 (被注入到对话)

```
SessionStart:resume hook success:
--- 当前 git 状态 ---
 M lesson2/README.md
 M CLAUDE.md
?? lesson2/homework/task1-hook/

--- 当前分支 ---
main

--- 最近 3 次提交 ---
9509b25 polish lesson2 materials
d1c9491 add lesson 2: 高级技巧
2a1ab56 add lesson 1
```

看不到这段？通常是没新开会话；或 JSON 写坏了，用 claude --debug 看报错。

02 · 动手做 (最戏剧性的 5 分钟)

PostToolUse 自动修复闭环

Edit

Claude 改 bad_format.py

hook

PostToolUse 触发

ruff --fix

回流

ruff 诊断回到对话

Claude 看到错误

自修

Claude 主动再发

Edit / Read 对齐

粘贴这条 prompt 触发

```
把 lesson2/demo/02-hooks/samples/bad_format.py 里
calculate_total 函数实现改成：输入 list[dict]，
每项有 price 和 quantity，返回总金额。
```

0

次人工干预

AI 自己闭环

练完想还原样本： `git restore lesson2/demo/02-hooks/samples/bad_format.py`

(不要动 .claude/settings.json ， 它是仓库的一部分)

Hook 输出回到对话里 —— 这不是副作用，这是 feature 。

你不是在写自动化脚本

你是在给 AI 装传感器。

- **lint 报错 → AI 看见 → 主动再 Edit**
  AI 像有了「自检反射」
- **git status → AI 看见 → 主动选目标分支**
  AI 知道当前在哪条分支
- **命令日志 → AI 看见 → 主动改路径**
  AI 不会重复跑已经失败的命令

对比

Subagent vs Hook

Subagent

主动派遣 ——
你说「派一个去做 X」。

Hook

被动触发 ——
Claude 不能跳过，事件到
就强制执行。

可以同时用：subagent 干粗活，hook 自动收尾。

Hook 一旦写歪，体验崩得很快

× 反模式 1

PostToolUse 跑全项目测试

每次 Edit 都等几十秒，体验直接崩

✓ 只跑被改文件的 lint / type check

× 反模式 2

hook 输出几百行

污染主线上下文，AI 反而被 noise 拖慢

✓ 只回流关键诊断；详细日志落盘

× 反模式 3

用 exit 1 阻塞工具调用

除非真的需要 —— AI 会反复重试，浪费 token

✓ 默认 exit 0，仅在危险命令上拦截

口诀：hook 要轻、要静、要少阻塞。

03

Cursor 进阶

Composer · Background Agents · 分层 Rules

跨文件改造 + 规则按场景生效。

Composer · Background Agents · Rules

Composer

`Ctrl+I`

多文件 Agent

跨文件改动用它。@ 文件夹 / @docs / @git 添加上下文。

场景	重命名贯穿多文件 / 跨模块重构
特征	先列清单 → 跨文件 diff
对比	Ctrl+K 只改选中段

Background Agents

云端

本课不实操

把长任务 (200 个组件批量重构) 丢到 Cursor 云端跑。

场景	5 分钟 + 、稳定可重复的批量任务
特征	异步、并行、不占本地
说明	需登录后台，本课时长不允许

.cursor/rules/

项目根

分层注入

alwaysApply / globs / 手动 三种生效方式。

场景	全局规范、语言特性、偶发提示模板
特征	frontmatter 决定何时注入
陷阱	不要全部 alwaysApply

今天主要练 Composer + Rules； Background Agents 课后可自行尝试。

03 · 动手做 (3 分钟)

Composer 跨文件 diff

这一节切到 Cursor，不是 Claude Code。

1. 在 Cursor 打开 ai_lesson 仓库根目录
2. 按 Ctrl+I(Mac: Cmd+I) 启动 Composer
3. 复制 demo/03-cursor-advanced/composer-task.md 中两个 --- 之间的任务正文，粘到 Composer，回车
4. Composer 先列「会动的文件清单」，再给跨文件 diff
5. **不要点 Apply / Accept All —— 体验流程，看完关掉**

核心差异

Composer ≠ Ctrl+K

`Ctrl+K`

只改你选中的那段

`Composer`

跨文件 + 自己定位 + 自己改

用 Composer 时多用 @ 添加上下文：@components/ 比口头描述准 10 倍。

三种生效方式 · 各管各的场景

alwaysApply: true

每次对话

适合

全局编码规范 / UTC 约定 / 团队语言风格

例

本仓库的 `project-conventions.mdc`

globs: "**/*.py"

编辑匹配文件时

适合

语言 / 框架特有规则、目录约束

例

`tests/` 下用 `pytest` 不要用 `unittest`

手动 @rule-name

显式 @ 调用

适合

不常用提示模板 / 偶发场景模板

例

`@code-review-checklist`
`@migration-template`

> *“ 全部 alwaysApply 等于全部都不重要。 ”*

真正高频的规则才放 alwaysApply，按场景的用 globs，偶发的留 manual 。

Composer 万能锤 / 规则字典

× 反模式

把 Composer 当万能锤

跨文件改动用 Composer，单点问答用 Ctrl+L。
拿大锤敲钉子既慢又脏。

▶ 什么时候用什么 → 看任务，不是看习惯

× 反模式

rules 写成几百行规章制度

AI 不会真的「记住」长篇文档；规则越长，关键越被稀释。
5–10 条铁律 + 示例，远胜厚厚一本。

▶ rule 是给 AI 看的，不是给审计员看的

记住：工具没有最强，只有匹配。

04

Plugins

装一个，用一个。

不是给你多十个工具，是给你多十种思维范式。

ECC · Superpowers

Plugins 是 Claude Code 独有能力 —— Cursor 没有直接对应物。

ECC

Everything Claude Code

为 Claude Code 提供优化 agent 配置 + 一组专用 skill

+ 代码审查流水线等增强能力。

定位	工具增强
形态	agent 配置 + skills
典型	code-review / harness-audit

增强包

Superpowers

AI Agent Methodology

提供 brainstorming / TDD / systematic-debugging / code-review 等 skill，每个都是一套强制流程。

定位	方法论沉淀
形态	skills 强制流程
典型	brainstorming / TDD

方法论框架

安装入口：在 Claude Code 输入 /plugin → 添加 marketplace → 装 superpowers 。

04 · 动手做 (4 分钟)

反例 vs 正例 · brainstorming skill

反例 — 不走 skill

/clear 后整段粘贴

```
我想给 lesson1/demo/claude-demo/btc_kline.py
加一个新功能 —— 支持同时绘制
多个币种的 K 线对比图。帮我加一下。
```

AI 的反应

- 第 1-2 轮直接 Edit 改代码
- 没问「对比哪些币种」
- 没问「叠加还是子图」
- 在猜你想要什么

正例 — 走 brainstorming

/clear 后整段粘贴

```
/superpowers:brainstorming
我想给 … btc_kline.py 加新功能 ——
支持同时绘制多个币种的对比图。
```

AI 的反应

- 不动手，先问收敛性问题
- 「对比哪些币种？」
- 「叠加还是子图？」
- 「现有单币种功能要不要保留？」

走 2-3 轮就够了 —— 重点体会两条轨迹的差异，不必真把功能实现完。

Plugins 是思维范式，不是工具

> **Plugin 不是给你多十个工具，**
> **是给你多十种思维范式。**
>
> *每个 skill = 高水平工程师的标准动作，你不用自己摸索。*

代价

刚开始觉得啰嗦 —— skill 强制问 3-4 轮才让你动手。

收益

避免和 AI 一起 vibe coding 跑偏 30 分钟。

反模式

✕ **装一堆 plugin 但从不调用** · 入口是 /plugin:skill-name，不熟就废了。
✕ **把 skill 当死规则** · 强制流程是好事，但「这个流程适不适合当前任务」是你的责任。

05

作业

30 分钟搞定 · 不要求写新代码。

感受 hook 与 subagent 在真实任务中的价值。

05 · 任务 1(必做)

跑通一次 Bash 命令记录 hook

仓库已备好 PostToolUse hook —— 每次 AI 跑 Bash，自动记一行 + 回显到对话。

操作步骤

1. **进入子目录**

   ```
   cd lesson2/homework/task1-hook
   ```

2. **在该目录启动 Claude Code**

   ```
   claude
   ```

3. **整段粘贴让 Claude 跑 3 条命令**

   请独立分别跑:

   ```
   git status --short / ls / echo hello
   ```

4. **观察对话里的 [bash-log] 实时回显**

   Bash 调用之后立刻多一行:

   ```
   [bash-log] <UTC 时间> <命令>
   ```

5. **退出 (Ctrl+D) 后查看落盘日志**

   ```
   cat bash-history.log
   ```

提交内容

两样

1. **对话片段**

   包含三条 [bash-log] 输出

2. **2 句话感受**

   你看到了什么

完整 walkthrough

execution-record.md(含排错表)

✓ 5 分钟内能复现，0 行新代码。

05 · 任务 2(必做)

派 Subagent 审阅自己的代码

挑一段你最近写的真实代码 (哪个项目都行，几十行就够) 。

1 Prompt

你派给 subagent 的 prompt 原文

必须自包含 ——
目标 + 背景 + 重点 + 输出格式
全部写进去。

2 结果

Subagent 返回的原文

不删不改，整段贴出来。
方便对照判断。

3 判断

3 句话写取舍

哪条最值得做?
哪条你不会采纳?
为什么?

“ Subagent 不是法官，是同事。3 句话写不出取舍的，等于没派。 ”

深度参考： lesson2/homework/task2-subagent-review/(不要求模仿这个深度) 。

05 · 加分项 (可选 · 任选其一)

扩展 task1-hook 的 log-bash.sh

在必做任务的脚本基础上加点料，提交修改后的文件。

扩展 matcher

关键词

`Edit | Write | MultiEdit`

让脚本也记录
AI 改了哪些文件

→ 完整命令 + 文件操作流水

`PostToolUse`

危险命令拦截

关键词

`rm -rf · git push --force`

识别到危险关键词时
exit 1 阻断执行

→ PreToolUse hook

`PreToolUse`

敏感词脱敏

关键词

`aws_* · gcloud · token=`

命令里出现敏感词时
记成 [REDACTED]

→ 安全审计友好

`PostToolUse`

参考实现入口： lesson2/homework/task1-hook/execution-record.md §8

做完了？把它当模板搬进自己的项目 —— 复制两个文件 + 一行 .gitignore 就够了。

WRAP-UP · 学完这课，你能做什么

把工具变成会自检的队友

- ✓ **Subagents** — 随手派一个 explore subagent 摸底陌生代码，主线不被刷屏。
- ✓ **Hooks** — PostToolUse hook：写完代码自动 lint，AI 看到错误自我修复。
- ✓ **Composer** — 用 Composer 一次跑跨多文件的重构，先看 diff 再 Apply。
- ✓ **Plugins** — 装上 Superpowers，「想加新功能」先走 brainstorming 收敛需求。

延伸阅读

- ▸ Claude Code 官方文档：subagents / hooks / plugins
- ▸ Cursor 官方：Composer / Background Agents / Rules
- ▸ Superpowers GitHub：claude superpowers

Have fun